

IDOR UYGULAMA VE ANALİZ RAPORU

SAYFA 1 — KAPAK SAYFASI

T.C. # ERCİYES ÜNİVERSİTESİ ## MÜHENDİSLİK FAKÜLTESİ ###
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

IDOR UYGULAMA VE ANALİZ RAPORU ## (Insecure Direct Object Reference
— Güvensiz Doğrudan Nesne Referansı) ### Zafiyet Simülasyonu, Sömürü
Süreci ve Güvenli Kod Geliştirme

HAZIRLAYANLAR | Öğrenci No | Ad Soyad | |:---:|:---:| | 1030521387 | Berkay
AYDEMİR | | 1030521205 | Fatih SARIDUMAN |

DANIŞMAN **Doç. Dr. Nesibe YALÇIN** *AVESİS — Erciyes Üniversitesi*

HAZİRAN 2026 **KAYSERİ**

SAYFA 2 – İÇİNDEKİLER VE ÖZET

İÇİNDEKİLER

Bölüm	Konu	Sayfa
1	Kapak Sayfası	sf. 1
2	İçindekiler ve Özet (Abstract)	sf. 2
3	Giriş ve IDOR Zafiyeti Teorik Altyapısı	sf. 3
4	Kullanılan Teknolojiler ve Laboratuvar Ortamı	sf. 4
5	Uygulama Mimarisi ve Senaryo Tasarımı	sf. 5
6	Zafiyetin Tespiti (Discovery Phase)	sf. 6
7	Zafiyetin Sömürülmesi (Exploitation Phase)	sf. 7
8	Etki Analizi ve Risk Değerlendirmesi	sf. 8
9	Çözüm ve Güvenlik Önerileri (Remediation)	sf. 9
10	Sonuç ve Kaynakça	sf. 10

ÖZET (ABSTRACT)

Web tabanlı uygulamaların yaygınlaşmasıyla birlikte, kullanıcı verilerinin gizliliği ve bütünlüğü, yazılım mühendisliği disiplininin en kritik araştırma konularından biri haline gelmiştir. OWASP API Security Top 10 listesinin birinci sırasında yer alan **Broken Object Level Authorization (BOLA / IDOR)** zafiyeti, modern web ve API mimarilerinde en sık karşılaşılan ve en yüksek etki potansiyeline sahip erişim kontrolü açıklarından biridir. Bu çalışma kapsamında, Python Flask çatısı kullanılarak gerçek bir dijital fatura yönetim sistemini birebir simüle eden eğitim amaçlı bir REST API uygulaması geliştirilmiştir. Geliştirilen uygulama, aynı uç noktayı (/api/invoice/<id>) tek tuşla **VULNERABLE (zafiyetli)** ve **SECURE (güvenli)** modlar arasında geçişlendirebilen çift modlu bir mimariye sahiptir; böylece zafiyetin sömürülmesi ve giderilmesi yan yana, canlı biçimde gözlemlenebilmektedir. Çalışmada, "Ahmet" kullanıcısı kimliğiyle giriş yapan bir saldırganın HTTP isteğindeki `invoice_id` parametresini manipüle ederek "Mehmet" kullanıcıasına ait 2001 ve 2002 numaralı faturalara yetkisiz erişim sağlama senaryosu kurgulanmış; saldırı, **Burp Suite Community Edition** proxy aracının **Repeater** modülü kullanılarak adım adım icra edilmiştir. Zafiyet, oturum bağlamındaki `user_id` ile kaynağın `owner_id` alanını her istekte karşılaştıran nesne-seviyesi sahiplik doğrulama mekanizması (`check_object_ownership`) eklenerek tamamen kapatılmıştır. Bunun yanında

parola özetleme (Werkzeug PBKDF2), oran sınırlama (rate-limiting), güvenlik başlıkları (CSP, X-Frame-Options) ve oturum sertleştirme gibi tamamlayıcı OWASP API2/API4/API5 önlemleri uygulanmıştır. Elde edilen sonuçlar, IDOR zafiyetlerinin çözümünde mimari düzeyde yetkilendirme tasarımının kaçınılmaz olduğunu ve "Authentication $\neq$ Authorization" ilkesinin yazılım yaşam döngüsünün her aşamasında gözetilmesi gerektiğini açıkça ortaya koymaktadır.

Anahtar Kelimeler: IDOR, BOLA, OWASP API Top 10, Broken Access Control, Flask, Burp Suite, Yetkilendirme, Güvenli Yazılım Geliştirme.

SAYFA 3 – GİRİŞ VE IDOR ZAFİYETİ TEORİK ALTYAPISI

3.1. Giriş

Bilgi teknolojilerinin endüstri ölçeğinde yaygınlaşması, web ve API tabanlı sistemlere duyulan güveni doğrudan bu sistemlerin güvenlik olgunluğuna bağımlı hale getirmiştir. Açık Web Uygulama Güvenliği Projesi (Open Web Application Security Project — OWASP), 2003 yılından bu yana web uygulamalarında en yaygın görülen ve en yüksek etkiye sahip güvenlik zafiyetlerini "Top 10" listeleri ile kamuoyuna duyurmaktadır. 2021 yılında güncellenen OWASP Top 10 listesinde **A01:2021 — Broken Access Control** kategorisi, önceki yıllarda beşinci sırada yer alan "kırık erişim kontrolü" başlığını birinciliğe taşımış; OWASP'ın 2023 API güvenlik listesinde ise bu kategorinin alt kümesi olan **API1:2023 — Broken Object Level Authorization (BOLA)** zafiyeti, tüm API'lerin en yaygın ve en kritik açığı olarak işaretlenmiştir.

Broken Access Control, kullanıcıların yalnızca sahip oldukları yetkiler dahilinde kaynaklara erişmesini sağlayan politikaların yazılım katmanında doğru uygulanamaması durumudur. Bu kategorinin en somut, en kolay sömürülebilen ve en sık karşılaşılan örneği ise **Insecure Direct Object Reference (IDOR — Güvensiz Doğrudan Nesne Referansı)** zafiyetidir. Bu çalışmanın konusu, IDOR zafiyetini teorik olarak açıklamak, gerçek bir senaryo üzerinde simüle etmek, Burp Suite kullanarak adım adım sömürmek ve aynı uygulama üzerinde güvenli yazılım pratikleriyle nasıl kapatılabileceğini göstermektir.

3.2. IDOR Nedir?

IDOR; bir uygulamanın iç kaynaklarına (örneğin bir fatura, bir profil, bir doküman veya bir kullanıcı kaydı) doğrudan, kullanıcı tarafından öngörülebilir veya manipüle edilebilir referanslar üzerinden erişim verirken, bu erişimin istemcinin (kullanıcının) yetkisi dahilinde olup olmadığını sunucu tarafında doğrulamamasıyla oluşan bir yetkilendirme zafiyetidir. Teknik mekanizma şu şekilde işler: istemci, bir HTTP isteği aracılığıyla sunucudan kaynak talep ederken, kaynak referansını (genellikle URL'nin yol parametresi, sorgu dizesi parametresi, JSON gövde alanı veya hatta çerez içeriği olarak) doğrudan gönderir. Sunucu, isteği yapan kişinin kimliğini (authentication) oturum çerezi ya da JWT üzerinden doğru biçimde okuyabilir; ancak istenen kaynağın referans değerine bu kullanıcının erişim hakkı olup olmadığını (authorization) kontrol etmez veya eksik kontrol eder. Bunun sonucunda saldırgan, kendi oturumunun geçerliliği bozulmaksızın, başka bir kullanıcıya ait kaynak referansını tahmin edip (örneğin ardışık sayısal ID'lerde 1001 → 2001 yaparak) o kullanıcıya ait veriye, dosyaya veya işleve ulaşır.

HTTP istek manipülasyonu, IDOR'un temel saldırı vektörüdür. Saldırgan; tarayıcının geliştirici araçlarını, bir HTTP istemcisini (cURL, Postman) ya da bir

araya getirici proxy aracı olan **Burp Suite**'i kullanarak istek paketini yakalar, kaynak referansını değiştirir ve isteği tekrar gönderir. Sunucu, isteği yapan kimliği değil; isteğin içindeki nesne referansının sahibini doğrulamak zorundadır. Aksi takdirde, geliştiricinin "bu URL'yi yalnızca sahibi bilir" varsayımı, yetkilendirmenin yerini alır ki bu, **security through obscurity (gizlilikle güvenlik)** olarak bilinen ve modern güvenlik mühendisliğinde reddedilen bir yanlış kabuldür.

3.3. Yatay ve Dikey Yetki Yükseltme Farkı

Erişim kontrolü ihlalleri literatürde iki ana eksende sınıflandırılır:

(a) Yatay Yetki Yükseltme (Horizontal Privilege Escalation): Saldırganın, kendisiyle **aynı rol seviyesindeki** başka bir kullanıcının verilerine veya kaynaklarına erişmesidir. Bu çalışmada simüle edilen senaryo birebir bu kategoriye aittir: "ahmet" rolünde bir normal kullanıcı (`role = "user"`), `/api/invoice/1001` ve `/api/invoice/1002` faturalarına meşru olarak erişebilirken, aynı seviyedeki "mehmet" kullanıcısına ait `/api/invoice/2001` ve `/api/invoice/2002` faturalarına IDOR yoluyla erişmektedir. Saldırgan rol değiştirmez; yatay bir komşusunun verisine sızar.

(b) Dikey Yetki Yükseltme (Vertical Privilege Escalation): Saldırganın **daha üst rol seviyesindeki** bir kullanıcıya (genellikle yöneticiye) ait fonksiyonlara veya verilere erişmesidir. Geliştirdiğimiz uygulamadaki `/admin/toggle-mode` ve `/admin/reset-stats` uç noktaları bu eksenin laboratuvarıdır. Eğer bu uçlar yalnızca `@login_required` ile korunsaydı, normal bir kullanıcı sistem davranışını değiştirebilir veya istatistikleri sıfırlayabilirdi. Bu, OWASP'ın **API5:2023 — Broken Function Level Authorization (BFLA)** kategorisi altında değerlendirilir ve uygulamamızda `@admin_required` dekoratörüyle kapatılmıştır.

İki eksen birbirinin tamamlayıcısıdır: nesne seviyesinde sahiplik kontrolü yatay yetki yükseltmeyi engellerken, fonksiyon seviyesinde rol kontrolü dikey yetki yükseltmeyi engeller. Olgun bir uygulama, her iki kontrolü bağımsız katmanlar olarak işletmek zorundadır.

SAYFA 4 — KULLANILAN TEKNOLOJİLER VE LABORATUVAR ORTAMI

4.1. Teknoloji Yığını

Aşağıdaki tablo, projede kullanılan tüm yazılım bileşenlerini, ilgili teknolojilerini ve uygulama içindeki rollerini göstermektedir.

Bileşen	Teknoloji	Kullanım Amacı
Backend Programlama Dili	Python 3.10+	API servisinin, yardımcı güvenlik modüllerinin ve test betiklerinin yazıldığı çekirdek dil.
Web Çerçevesi	Flask 3.0+	REST API uç noktalarının (/login, /api/invoice/<id>, /admin/...) tanımlanması ve HTTP istek/cevap akışının yönetilmesi.
Şablon Motoru	Jinja2 (Flask varsayılan)	Saldırgan ve sunucu panellerinin yan yana sunulduğu interaktif demo arayüzünün (templates/index.html) sunucu tarafından üretilmesi.
Parola Özetleme	Werkzeug (generate_password_hash, check_password_hash)	Kullanıcı parolalarının düz metin yerine PBKDF2 tabanlı tek yönlü özet (hash) formatında saklanması; sabit-zamanlı doğrulama.
Oturum Yönetimi	Flask session (imzalı çerez)	Giriş yapan kullanıcının user_id, username ve role bilgilerini sunucu tarafında oturum boyunca güvenli biçimde taşımak.
Veritabanı (Demo)	Python sözlüğü (in-memory) — USERS, INVOICES	Eğitim amaçlı izole demo için bellek içi veri saklama; gerçek RDBMS yerine ardışık tam sayı ID'lerin IDOR için ideal hedefi olduğunu net göstermek.
Yetkilendirme Mekanizması	Özel dekoratörler: login_required,	Authentication ve Authorization katmanlarının açıkça birbirinden ayrılması; sahiplik

Bileşen	Teknoloji	Kullanım Amacı
	<code>admin_required</code> , <code>owner_required</code>	kontrolünün merkezi olarak uygulanması.
Oran Sınırlama (Rate Limiting)	Özel <code>RateLimiter</code> (sliding window, bellek içi)	OWASP API4'e uygun olarak / login üzerindeki kaba kuvvet (brute-force) saldırılarının IP başına 60 saniyede 5 deneme ile sınırlandırılması.
Güvenlik Loglaması	Python <code>logging</code> modülü, <code>idor.security</code> adlı özel logger	Yetkisiz erişim girişimlerinin (kullanıcı, kaynak, IP) zaman damgalı kayıt altına alınması; gerçek dünyada SIEM (Splunk, ELK) entegrasyonuna hazır biçim.
Test Çerçevesi	Python <code>unittest</code> + Flask test istemcisi (<code>test_cases.py</code>)	31 adet birim testi ile authentication, IDOR (her iki mod) ve API sertleştirmelerinin otomatik olarak doğrulanması.
Saldırı Aracı (Proxy)	Burp Suite Community Edition 2024.x	HTTP isteklerinin yakalanması, Repeater modülü ile manipüle edilmesi ve sunucu cevaplarının analiz edilmesi.
Yardımcı Saldırı Aracı	Python <code>requests</code> kütüphanesi (<code>exploit_demo.py</code>)	Terminal üzerinden otomatik IDOR taraması ve renkli adım adım sömürü demonstrasyonu.
Tarayıcı Geliştirici Araçları	Chromium / Firefox DevTools — Network sekmesi	İlk keşif aşamasında uygulama trafiğinin pasif olarak gözlemlenmesi ve <code>invoice_id</code> parametresinin tespiti.
Konteynerizasyon (opsiyonel)	Docker (lokal izolasyon)	Demo sunucunun ana işletim sisteminden izole bir konteynerde çalıştırılabilmesi için temel altyapı.
Sürüm Kontrolü	Git + GitHub (<code>aydemirbrkay/ IDOR</code>)	Kaynak kodun versiyonlanması, akademik teslim edilebilirlik ve <code>claude/brave-cray-2vuem3</code> üzerinde geliştirme dalı kullanımı.

4.2. Laboratuvar Ortamının İzolasyonu

Geliştirme ve sömürü deneyleri, kontrollü bir laboratuvar ortamında, kamuya açık herhangi bir ağa erişim olmaksızın gerçekleştirilmiştir. Flask geliştirme

sunucusu, varsayılan olarak yalnızca yerel makinede dinlenen `http://localhost:5000` adresine bağlanmaktadır. Demo amacıyla `host="0.0.0.0"` ile geliştirilse de testlerin tamamı, dış IP adresine maruz kalmayan tek bir geliştirici makinesi üzerinde icra edilmiştir. Bu sayede, eğitim amaçlı bilinçli olarak bırakılan zafiyetin (VULNERABLE mod) üçüncü taraflarca sömürülmesi engellenmiştir.

İsteğe bağlı izolasyon için sunucu, hafif bir **Docker** konteyneri içerisinde de çalıştırılabilmekte; bu mimaride uygulama, ana makinenin işletim sistemi kaynaklarından konteyner sınırlarıyla ayrılır ve port yönlendirme (`-p 5000:5000`) yalnızca `localhost` arayüzüne yapılır. Saldırgan rolündeki Burp Suite ise aynı makine üzerinde `127.0.0.1:8080` proxy adresinde dinleyerek, tarayıcı trafiğini ortadaki adam (man-in-the-middle) konumundan yakalamakta ve sömürü adımlarını **Repeater** modülünde yeniden inşa etmektedir. Tüm test kullanıcıları (`ahmet`, `mehmet`, `admin`) ve test fatura kayıtları (`#1001`, `#1002`, `#2001`, `#2002`) yalnızca bu izole ortamda mevcut olup, gerçek kişisel veri içermemektedir; böylece KVKK ve etik araştırma ilkelerine tam uyum sağlanmıştır.

[GÖRSEL EKLE: Çalışan Flask Sunucusunun Terminal Çıktısı] — `python app.py` komutunun yürütülmesinden sonra terminalde görünen "[IDOR Sunucu] `http://localhost:5000` — Mod: ZAFİYETLİ" mesajı ve Flask'ın başlangıç bannerını içeren ekran görüntüsü eklenmelidir.

SAYFA 5 — UYGULAMA MİMARİSİ VE SENARYO TASARIMI

5.1. İş Mantığı (Business Logic)

Geliştirilen uygulama, bir dijital muhasebe / fatura yönetim sisteminin minimum çekirdek işlevselliğini simüle etmektedir. Sistem üç ana aktöre hizmet verir: iki normal son kullanıcı (ahmet ve mehmet) ve bir yönetici (admin). Her son kullanıcı, kendi hesabına bağlı birden fazla faturayı (INVOICES sözlüğü) görüntüleyebilir; yönetici ise sistemin çalışma modunu (zafiyetli/güvenli) değiştirebilir ve istatistikleri sıfırlayabilir. İş mantığının çekirdek varlığı **Fatura (Invoice)** nesnesidir; her fatura yapısal olarak bir benzersiz id, sahibinin owner_id'si, sahibinin adı (owner), hizmet türü, tutar, tarih ve ödeme durumu alanlarından oluşmaktadır.

Kullanıcı oturum süreci şu şekilde işler: istemci, /login uç noktasına POST metodu ile kullanıcı adı ve parolayı JSON gövdesinde gönderir; sunucu, parolayı Werkzeug ile hash karşılaştırması üzerinden doğrular, başarılıysa oturum çerezini yeniler (session fixation savunması), user_id, username ve role alanlarını oturuma yazar ve bir karşılama yanıtı döner. Bu noktadan sonra istemci, /api/my-invoices uç noktasıyla kendi faturalarını listeleyebilir veya /api/invoice/<int:invoice_id> üzerinden tekil bir faturayı görüntüleyebilir. İşte tam bu ikinci uç nokta, çalışmanın merkezindeki **IDOR noktasıdır**.

5.2. Veritabanı Tasarımı ve ID Stratejisi

Demo veritabanı, eğitim amacının gerektirdiği saydamlık nedeniyle bellek içi Python sözlükleri olarak kurgulanmıştır. Fatura ID'leri kasıtlı olarak **ardışık ve öngörülebilir tam sayılar** olarak seçilmiştir (1001, 1002, 2001, 2002). Bu seçim iki amaca hizmet eder:

1. **Saldırı ufkunu görünür kılmak:** ardışık tam sayılar, saldırganın "bir sonraki ID'yi tahmin et" stratejisini tetikleyen klasik IDOR koşulunu oluşturur. Gerçek dünya vakalarının önemli bir bölümünde (Snapchat, Parler, T-Mobile vakaları dahil) zafiyetin doğrudan kaynağı, otomatik artıslı (auto-increment) birincil anahtarların API URL'lerine doğrudan yansıtılmasıdır.
2. **Savunma katmanını öğretmek:** sahiplik kontrolü, ID'lerin tahmin edilebilirliğinden bağımsız olarak çalışmak zorundadır. Bu sayede "UUID kullanırsam IDOR olmaz" yanılgısı, mimari bir karşıloluğ üzerinden çürütülmektedir.

USERS sözlüğü kullanıcı bilgilerini, INVOICES sözlüğü ise fatura kayıtlarını tutar. Faturalar ile kullanıcılar arasında owner_id alanı üzerinden bire-çok (one-to-many) ilişki bulunur. Bu ilişki, sahiplik kontrolünün dayanağıdır: her erişim

isteğinde sunucu, "isteği yapan kullanıcının `session["user_id"]` değeri ile istenen kaynağın `owner_id` değeri eşleşiyor mu?" sorusunu yanıtlamak zorundadır.

5.3. Saldırı Senaryosunun Kurgusu

Senaryomuz, gerçek dünyada en sık karşılaşılan IDOR vakalarını birebir yansıtacak biçimde kurgulanmıştır:

Aktörler: - **Saldırgan:** "ahmet" (`user_id = 1`, parola: `ahmet123`) — sisteme meşru bir hesapla giriş yapan kötü niyetli bir abone. - **Kurban:** "mehmet" (`user_id = 2`) — saldırganla aynı yetki seviyesindeki başka bir abone.

Saldırı Akışı: 1. **Adım 1 — Meşru Giriş:** Saldırgan, web arayüzü üzerinden kendi gerçek kimlik bilgileriyle (`ahmet / ahmet123`) `POST /login` isteğini gönderir. Sunucu, geçerli oturum çerezi (`session`) atar. Bu noktada saldırganın yetkisi, oturum bağlamına göre yalnızca `1001` ve `1002` numaralı faturalarla sınırlıdır. 2. **Adım 2 — Meşru Kaynak Erişimi:** Saldırgan, "Faturalarım" düğmesine tıklayarak `GET /api/my-invoices` ve ardından `GET /api/invoice/1001` isteklerini üretir. Cevap olarak Ahmet'in 2.500 TL'lik "Web Tasarım Hizmeti" faturası döner. Bu meşru istek, ID parametresinin URL yolunda bulunduğunu gözler önüne serer. 3. **Adım 3 — Parametre Manipülasyonu:** Saldırgan, tarayıcı geliştirici araçlarındaki Network sekmesinde veya Burp Suite proxy üzerinde isteği yakalar, URL'deki `1001` değerini farklı bir kullanıcının fatura kimliğine (örneğin `2001`) değiştirir ve aynı oturum çerezi ile yeniden gönderir. 4. **Adım 4 — Yetkisiz Veri İfşası:** Sunucu **VULNERABLE** modda sahiplik kontrolü yapmadığından `200 OK` cevabıyla Mehmet'in 9.750 TL'lik "Yazılım Geliştirme" faturasını saldırganı iade eder. Saldırgan, kendi oturumunu hiç bozmadan, hiçbir alarmı tetiklemeden, yalnızca URL'deki bir sayıyı değiştirerek başka bir kullanıcının hassas finansal verisine ulaşmıştır. 5. **Adım 5 — Sistematik Tarama (Otomasyon):** Saldırgan bir adım daha ileri giderek `exploit_demo.py` benzeri bir betikle `1000–3000` arasındaki tüm fatura ID'lerini sıralı biçimde dener ve sistemdeki **tüm faturaları** dökmüş olur.

[GÖRSEL 1: Uygulama İşleyiş ve İstek Akış Şeması] — Saldırgan tarayıcısı → Burp Suite Proxy (`127.0.0.1:8080`) → Flask uygulaması (`localhost:5000`) → bellek içi `INVOICES` sözlüğü zincirini gösteren, üzerinde "Adım 1: Giriş", "Adım 2: Meşru İstek", "Adım 3: Manipüle Edilmiş İstek", "Adım 4: Yetkisiz Cevap" oklarının bulunduğu bir akış diyagramı eklenmelidir. Diyagramda **VULNERABLE** ve **SECURE** modlarının ayrımı renk kodlamasıyla belirtilmelidir.

SAYFA 6 — ZAFİYETİN TESPİTİ (DISCOVERY PHASE)

6.1. Pasif Keşif: Trafik Gözlemi

Bir uygulamada IDOR avına çıkan saldırgan veya beyaz şapkalı güvenlik araştırmacısı, ilk aşamada **pasif keşif** yapar; yani uygulamayı meşru bir kullanıcı gibi kullanarak üretilen HTTP trafiğini gözlemler. Bu çalışmada keşif süreci aşağıdaki sırada ilerlemiştir:

İlk adımda Chromium tabanlı tarayıcı (Brave / Google Chrome) üzerinde F12 ile Geliştirici Araçları açılmış ve **Network (Ağ)** sekmesi etkinleştirilmiştir. Saldırgan rolündeki "ahmet" kullanıcısı, web arayüzü üzerinden meşru giriş yaptığında Network sekmesinde aşağıdaki istek dizisi gözlemlenmiştir:

1. POST /login — gövdesinde {"username":"ahmet","password":"ahmet123"} taşıyan istek. Yanıt: 200 OK ve sunucudan dönen Set-Cookie: session=... başlığı.
2. GET /api/my-invoices — kullanıcının kendi faturalarını döndüren istek.
3. GET /api/invoice/1001 — kullanıcının fatura detayını görüntülemek için yaptığı istek.

Bu üçüncü istek, IDOR avcısının dikkatini çekecek ilk kritik sinyaldir: **kaynak referansı (1001) doğrudan URL yolunda görünmektedir**, başka bir deyişle istemci tarafından üretilen ve sunucu tarafından sorgusuz kabul edilen bir tanımlayıcıdır. URL yolundaki bu sayısal değer, IDOR için ilk hipotezi oluşturur.

6.2. Aktif Keşif: Burp Suite Proxy ile Yakalama

İkinci aşamada saldırgan, üretim sınıfı bir proxy aracına geçer. Bu çalışmada **Burp Suite Community Edition** kullanılmıştır. Burp Suite'in Proxy → Intercept modülü etkinleştirilmiş, FoxyProxy gibi bir tarayıcı uzantısı veya tarayıcı sistem ayarları aracılığıyla tüm HTTP/HTTPS trafiği 127.0.0.1:8080 üzerinden Burp'a yönlendirilmiştir.

Saldırgan tarayıcıda yeniden 1001 numaralı faturayı görüntüleyince Burp Suite isteği yakalar:

```
GET /api/invoice/1001 HTTP/1.1
Host: localhost:5000
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
Accept: application/json
Accept-Language: tr-TR,tr;q=0.9,en-US;q=0.8
Cookie: session=eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFobWV0Iiwicm9sZSI6InVzZXIifQ...
Connection: close
```

Bu yakalanan isteğin teknik analizinde üç bulgu öne çıkar:

1. **Kaynak Referansı:** URL yolunda yer alan 1001 değeri, doğrudan veritabanı birincil anahtarına karşılık gelmektedir. Bu, IDOR için en saf hedeftir.
2. **Oturum Çerezi:** Cookie: session=... başlığı, Flask'ın imzalı oturum çerezini taşır. Saldırgan, bu çerezi bozmadan yalnızca URL'yi değiştirerek aynı oturumla farklı bir kaynak isteyebilir; bu, sunucunun "kim?" sorusuna doğru cevap vereceği, ancak "neye?" sorusunu sormayacağı anlamına gelir — IDOR'un tetiklenmesi için gerekli koşul.
3. **Yetkilendirme Başlığının Yokluğu:** İstekte Authorization başlığı bulunmamaktadır; tüm yetkilendirme oturum çerezine indirgenmiştir. Bu durum, kaynak referansının yetkilendirme bağlamında değil, yalnızca kimlik bağlamında değerlendirileceği endişesini güçlendirir.

6.3. Hipotezin Doğrulanması

Saldırgan, yakalanan isteği Burp Suite'in Action → Send to Repeater seçeneği ile Repeater modülüne aktarır. Repeater'da URL'deki 1001 değerini 2001 ile değiştirip "Send" butonuna basar. Sunucu yanıtı aşağıdaki gibi döner:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 2001,
  "owner_id": 2,
  "owner": "Mehmet Kaya",
  "amount": "₺9.750,00",
  "service": "Yazılım Geliştirme",
  "date": "2024-01-20",
  "status": "Ödendi"
}
```

200 OK durum kodu ve gövdede dönen "Mehmet Kaya" verisi, IDOR hipotezini tam olarak doğrulamıştır. Saldırgan artık sistemde nesne seviyesinde bir yetkilendirme açığının varlığından emindir ve sömürü aşamasına geçer.

[GÖRSEL 2: Tarayıcı Network Sekmesi ve Burp Suite Proxy Üzerinde Yakalanan Orijinal İstek] — Solda Chrome/Firefox DevTools'un Network sekmesinde GET /api/invoice/1001 isteğinin görünür olduğu ekran görüntüsü; sağda Burp Suite Proxy → HTTP history sekmesinde aynı isteğin başlıklarıyla birlikte yakalanmış halinin ekran görüntüsü eklenmelidir. Oturum çerezi ve URL yolundaki 1001 değeri kırmızı çerçeve ile işaretlenmelidir.

SAYFA 7 — ZAFİYETİN SÖMÜRÜLMESİ (EXPLOITATION PHASE)

7.1. Burp Suite Repeater ile Manuel Sömürü

Sömürü fazı, bir önceki aşamada doğrulanan hipotezin sistematik olarak işletilmesini ve elde edilebilecek tüm yetkisiz verinin döküm alınmasını kapsar. Bu çalışmada sömürü, Burp Suite'in **Repeater** modülünde adım adım gerçekleştirilmiştir.

Adım 1 — İsteğin Repeater'a Aktarılması: Proxy → HTTP history sekmesinde yakalanan GET /api/invoice/1001 isteği üzerine sağ tıklanmış, açılan menüden Send to Repeater (Ctrl+R) seçeneği işaretlenmiştir. Burp Suite, bu isteği orijinal başlıkları ve oturum çereziyle birlikte Repeater sekmesine taşımıştır.

Adım 2 — Temel Çizgi (Baseline) İsteğinin Gönderilmesi: Repeater sekmesinin sol panelinde isteğin orijinal hali yer almaktadır. Send butonuna ilk tıklayışta sağ panelde aşağıdaki yanıt görülür:

```
HTTP/1.1 200 OK
{ "id": 1001, "owner_id": 1, "owner": "Ahmet Yılmaz", "amount":
"₺2.500,00", ... }
```

Bu yanıt, saldırganın kendi meşru faturasıdır ve sömürü için bir temel çizgi (baseline) oluşturur.

Adım 3 — İlk IDOR Denemesi (Komşu Kullanıcı): Saldırgan, URL satırındaki 1001 değerini 2001 olarak değiştirir ve Send butonuna yeniden basar. Sunucudan dönen yanıt:

```
HTTP/1.1 200 OK
{ "id": 2001, "owner_id": 2, "owner": "Mehmet Kaya", "amount": "₺9.750,00",
"service": "Yazılım Geliştirme", "date": "2024-01-20", "status": "Ödendi" }
```

Bu, başka bir kullanıcının (Mehmet Kaya) finansal verisidir ve saldırganın kendi oturumu üzerinden hiçbir uyarı tetiklemeden iade edilmiştir. Saldırgan, **yatay yetki yükseltme** ile sistemin gizlilik kontrolünü ihlal etmiştir.

Adım 4 — İkinci IDOR Denemesi: URL 2002 olarak değiştirilir; Mehmet'in ikinci faturası (₺3.200,00 — "Sunucu Bakımı") ifşa olur.

Adım 5 — Sistemik Tarama (Intruder Senaryosu): Manuel deneme yerine otomatikleştirilmiş bir saldırı için aynı istek Send to Intruder ile Intruder modülüne aktarılabilir. URL'deki 1001 değeri Burp Intruder'da bir payload pozisyonu (§...§) olarak işaretlenir; payload türü olarak "Numbers" seçilir, başlangıç 1000, bitiş 3000, adım 1 yapılır. "Start attack" ile saldırı başlatıldığında Burp; her sayısal değer için tek tek istek gönderir, dönen Status

ve Length sütunlarını sıralamak suretiyle 200 OK dönen tüm meşru fatura ID'lerini tek bir tablo halinde dökmüş olur. Bu, sistem genelinde bir **mass enumeration** (kitlesel kaynak ifşası) saldırısıdır ve gerçek dünyada çoğu zaman tüm müşteri veritabanının dış sızıntısıyla sonuçlanır.

7.2. Otomatik Sömürü: `exploit_demo.py`

Çalışmanın bir parçası olarak Burp Intruder'ın muadili olan kısa bir Python istismar betiği (`exploit_demo.py`) da geliştirilmiştir. Bu betik; `requests.Session` ile saldırgan oturumu açar, ardından 1000–3000 aralığındaki ID'leri art arda dener ve 200 OK dönen tüm faturaları renkli biçimde terminale döker. Bu yaklaşım, IDOR zafiyetinin yalnızca tek tek manuel denemelerle değil, otomasyon ile saniyeler içinde tüm veritabanını dışarı çıkarabilecek bir saldırı vektörüne dönüştüğünü gösterir.

7.3. Sunucu Cevabının Analizi

VULNERABLE modda sunucunun döndürdüğü yanıt, normal bir başarılı yanıtla **birebir aynıdır**. Bu durum saldırgan için iki açıdan kritiktir:

1. **Yan kanal yokluğu:** Yanıt kodu ya da içerik şekli, IDOR'un sömürüldüğünü göstermez; uygulama saldırıya direnmek bir yana, saldırının başarılı olduğunu meşru bir cevapmış gibi onaylar.
2. **Loglama farkı:** VULNERABLE modda `app.py` içindeki `STATS["successful_exploits"]` sayacı arka planda artar ve `security_logger` bir bilgi (INFO) satırı yazar; ancak istemci tarafına bu uyarı yansımaz. Olay sonrası adli analiz olmadığı sürece sömürü fark edilmez. Gerçek üretim sistemlerinde bu davranış aylar süren veri sızıntılarının nasıl algılanamadığını doğrudan açıklar.

[GÖRSEL 3: Burp Suite Repeater Ekranında Değiştirilen Request ve Başarılı Dönen Yetkisiz Response] — Burp Suite Repeater penceresinin solunda GET `/api/invoice/2001` (önceden 1001'di) isteği, sağda ise 200 OK durum koduyla birlikte Mehmet Kaya'ya ait ₺9.750,00 tutarındaki faturanın JSON cevabı görünmelidir. URL'deki değiştirilen 2001 değeri ve cevap gövdesindeki `"owner": "Mehmet Kaya"` ifadesi sarı vurguyla işaretlenmelidir. Ek olarak, sunucu terminalinde aynı anda görülen [ZAFİYET] Kullanıcı 1 → Fatura 2001 (Mehmet Kaya) – Sahiplik kontrolü yok! log satırı da ikinci bir alt görsel olarak eklenebilir.

SAYFA 8 — ETKİ ANALİZİ VE RİSK DEĞERLENDİRMESİ

8.1. Sızdırılan Veri Sınıfı ve Hassasiyet Profili

Sömürü sonucunda saldırganın eline geçen veri kümesi, finansal bir uygulama bağlamında en hassas sınıfta yer almaktadır: faturanın ait olduğu **kişinin adı-soyadı, alınan hizmetin türü, fatura tutarı, fatura tarihi ve ödeme durumu**. Bu beş alan bir araya geldiğinde, üçüncü bir taraf için aşağıdaki çıkarımları yapma imkânı doğmaktadır:

- Bir kişinin hangi hizmet sağlayıcılarla ticari ilişkide bulunduğu (rakip istihbaratı).
- Müşterinin finansal hacmi ve ödeme alışkanlıkları (sosyal mühendislik için zemin).
- Ödenmemiş borçların varlığı (şantaj veya doğrudan dolandırıcılık vektörü).
- Müşteri portföyünün toplam büyüklüğü (kurumun gerçek gelirini hesaplama).

8.2. Yasal ve Düzenleyici Etkiler

Türkiye'de yürürlükte olan **6698 sayılı Kişisel Verilerin Korunması Kanunu (KVKK)** çerçevesinde fatura sahibinin adı, hizmet kalemi ve tutar bilgileri "kişisel veri" olarak kabul edilmektedir. Bir IDOR yoluyla bu verinin ifşa olması; KVKK madde 12 (veri güvenliğine ilişkin yükümlülükler) ve madde 18 (idari para cezaları) hükümlerinin doğrudan ihlali anlamına gelir. Kurum, ihlali en geç **72 saat içinde** Kişisel Verileri Koruma Kuruluna (KVKK Kurulu) bildirmek zorundadır; bildirilmemesi veya ihmal tespiti hâlinde idari para cezası **milyon TL** mertebelerine ulaşmaktadır. Avrupa Birliği müşterilerine hizmet veriliyor ise aynı zafiyet **GDPR Madde 32 ve 33** kapsamında değerlendirilir ve para cezası küresel cironun **%4'üne** kadar çıkabilmektedir.

8.3. Operasyonel ve İtibari Etkiler

Bir IDOR sızıntısının kuruma dolaylı etkileri, doğrudan para cezalarından çoğu zaman daha büyüktür:

- **Müşteri güveni kaybı:** Snapchat (2014) ve Parler (2021) gibi olaylarda IDOR ile dökülen verilerin etkisi, marka değeri üzerinde uzun vadeli erozyona yol açmıştır.
- **Hukuki dava maliyetleri:** Müşterilerin bireysel ya da toplu hukuki süreçler başlatması.
- **İş sürekliliği kaybı:** Olay müdahale (Incident Response) süreçleri sırasında sistemin geçici olarak kapatılması veya yetkilendirme modelinin yeniden tasarlanması.

- **Tedarikçi sözleşmelerinin feshi:** Kurumsal müşteriler, sözleşmelerinde "veri ihlali" şartı bulunan tedarikçilerden uzaklaşmaktadır.

8.4. Saldırı Senaryosunun CVSSv3.1 Vektörü ve Skoru

Uygulamamızdaki IDOR zafiyeti için **Common Vulnerability Scoring System (CVSS) sürüm 3.1** bağlamında aşağıdaki vektör çıkarımı yapılmıştır:

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

Metrik	Değer	Gerekçe
Attack Vector (AV)	Network (N)	Uç noktaya internet üzerinden erişilmektedir.
Attack Complexity (AC)	Low (L)	Saldırı, URL'deki sayısal bir değer değiştirilmesinden ibarettir; herhangi bir koşul gerektirmez.
Privileges Required (PR)	Low (L)	Saldırganın yalnızca standart bir kullanıcı oturumuna ihtiyacı vardır.
User Interaction (UI)	None (N)	Kurban kullanıcının herhangi bir aksiyon alması gerekmez.
Scope (S)	Unchanged (U)	Etki, aynı güvenlik bağlamı içindedir.
Confidentiality (C)	High (H)	Tüm fatura veritabanı sızdırılabilir; gizlilik tamamen ihlal edilir.
Integrity (I)	None (N)	Mevcut endpoint salt okunurdu; ancak PUT/DELETE genişletilirse bu metrik High olur.
Availability (A)	None (N)	Servis erişilebilirliği etkilenmez.

Hesaplanan Temel Skor: 6.5 – Orta (Medium)

Ancak veri sınıfı (kişisel ve finansal) ve gerçek dünya etkisi düşünüldüğünde, bu skorun **çevresel metrikler (Environmental Metrics)** ile ağırlandırılması zorunludur. CR:H (Confidentiality Requirement: High) uygulandığında skor **7.7 – Yüksek (High)** düzeyine çıkmaktadır. OWASP raporlarına göre BOLA/IDOR zafiyetlerinin **bug bounty** programlarında ödenen ortalama ödülü 3.000-10.000 USD arasındadır; yani sektör, bu zafiyetin gerçek dünya kritikliğini **Yüksek** olarak konumlandırmaktadır. Bu çalışmada da zafiyet, **Yüksek kritiklik seviyesi (High Severity)** ile değerlendirilmiş ve acil giderim (immediate remediation) önceliğine alınmıştır.

SAYFA 9 — ÇÖZÜM VE GÜVENLİK ÖNERİLERİ (REMEDIATION)

9.1. Mimari Düzeyde Çözüm: Nesne Seviyesi Sahiplik Kontrolü

IDOR zafiyetinin giderilmesinde tek geçerli ve kalıcı çözüm, her nesne erişiminde sunucu tarafında **"bu kaynak, bu oturuma gerçekten ait mi?"** sorusunun açıkça yanıtlanmasıdır. Bu çalışmada bu kontrol, `security_utils.py` modülünde tek bir merkezi fonksiyon olarak yazılmış ve uygulama genelinde tutarlı biçimde çağırılmıştır:

```
def check_object_ownership(resource: dict, current_user_id: int,
                           owner_field: str = "owner_id") -> bool:
    """Kaynağın belirli bir kullanıcıya ait olup olmadığını kontrol
    eder."""
    return resource.get(owner_field) == current_user_id
```

Bu fonksiyon mimari açıdan iki sebepten kritiktir: (1) sahiplik mantığı kod tabanında **tek bir noktada** tanımlıdır; bir hata düzeltildiğinde tüm endpoint'ler aynı anda korunur. (2) Endpoint yazarı, sahiplik kontrolünü unutsa bile `@owner_required` dekoratörü mevcut olduğundan kontrol **deklaratif** biçimde uygulanabilir, bu da insan hatasını minimize eder.

9.2. Hatalı / Zafiyetli Kod Bloğu (Before)

Aşağıdaki kod parçası, `app.py` dosyasında `SECURE_MODE = False` iken çalışan, **IDOR'a açık** yetkilendirme akışını temsil etmektedir:

```
@app.route("/api/invoice/<int:invoice_id>", methods=["GET"])
@login_required # Yalnızca kimlik doğrulama yapılır; yetkilendirme YOK
def get_invoice(invoice_id):
    invoice = INVOICES.get(invoice_id)

    if not invoice:
        return jsonify({"error": "Fatura bulunamadı."}), 404

    # ZAFİYET: Sahiplik kontrolü hiç yapılmıyor!
    # Giriş yapmış herhangi bir kullanıcı, herhangi bir faturayı görebilir.
    # `session["user_id"]` ile `invoice["owner_id"]` karşılaştırması YOK.

    return jsonify(invoice), 200 # Yetkisiz veri ifşası
```

Bu kodun temel problemi, `@login_required` dekoratörünün yalnızca "kim?" sorusunu yanıtlamasıdır; "ne?" sorusuna yanıt verilmediği için, geçerli bir

oturum çerezine sahip herhangi bir saldırgan, parametreyi değiştirerek istediği faturaya ulaşmaktadır.

9.3. Fixlenmiş / Güvenli Kod Bloğu (After)

Aşağıdaki kod parçası, `SECURE_MODE = True` iken çalışan ve IDOR'u **etkin biçimde engelleyen** sürümdür:

```
@app.route("/api/invoice/<int:invoice_id>", methods=["GET"])
@login_required # 1) Authentication: Önce kullanıcı oturumunu zorunlu kıl
def get_invoice(invoice_id):
    invoice = INVOICES.get(invoice_id)

    if not invoice:
        return jsonify({"error": "Fatura bulunamadı."}), 404

    current_user_id = session.get("user_id")

    # 2) Authorization: Nesne seviyesi sahiplik kontrolü
    # `check_object_ownership` merkezi fonksiyonu owner_id == user_id
    # eşitliğini doğrular. Sahiplik yoksa erişim REDDEDİLİR.
    if not check_object_ownership(invoice, current_user_id):
        # 3) Yetkisiz girişimi güvenlik loguna düşür (SIEM hazırlığı)
        log_unauthorized_access(
            user_id=current_user_id,
            resource_type="invoice",
            resource_id=invoice_id,
            ip=request.remote_addr,
        )
        # 4) Bilinçli 404: 403 "ID geçerli ama yetkin yok" bilgisini
        # sızdırır
        # 404 ile saldırganın enumeration yapması engellenir.
        return jsonify({"error": "Fatura bulunamadı."}), 404

    return jsonify(invoice), 200 # Sadece sahip kullanıcıya iade edilir
```

Bu çözümde dört savunma katmanı bir arada işletilmektedir:

1. **Kimlik doğrulama:** `@login_required` ile otursuz istek 401 ile reddedilir.
2. **Nesne seviyesi yetkilendirme:** `check_object_ownership` ile sahiplik karşılaştırması yapılır.
3. **Güvenlik loglaması:** İhlal girişimi `idor.security logger`'ı aracılığıyla zaman damgalı kayıt altına alınır; gerçek üretimde bu kayıt Splunk veya ELK'ya iletilebilir.
4. **Bilgi sızıntısının önlenmesi:** Yetkisiz erişimde **403** yerine **404** dönülerek saldırganın "bu ID mevcut" çıkarımı yapması engellenir; böylece kitlesel enumeration ek bir maliyetle çarpılır.

9.4. Dekorator Tabanlı Beyan Edici (Declarative) Çözüm

Endpoint başına manuel kontrol yazmak yerine, kontrolü dekoratörle merkezileştirmek daha sürdürülebilir bir yaklaşımdır:

```
@app.route("/api/invoice/<int:invoice_id>", methods=["GET"])
@login_required
@owner_required(lambda inv_id: INVOICES.get(inv_id)) # Sahiplik
dekoratörle
def get_invoice(invoice_id):
    return jsonify(INVOICES[invoice_id]), 200
```

Bu yaklaşım, kontrolü gözden kaçırma riskini ortadan kaldırır ve kod tabanındaki **erişim politikasını okunabilir** kılar.

9.5. Tamamlayıcı Sertleştirmeler

IDOR'un birincil çözümü sahiplik kontrolü olsa da, **savunma derinliği (defense in depth)** ilkesi gereği aşağıdaki tamamlayıcı önlemler de uygulanmıştır:

- **UUID Kullanımı (Önerilen Geliştirme):** Ardışık tam sayı ID'ler yerine `uuid.uuid4()` ile üretilen tahmin edilemez tanımlayıcılar, saldırganın kaba kuvvet enumeration maliyetini katlanarak arttırır. Ancak bu, **sahiplik kontrolünün yerine değil yanına** eklenmesi gereken bir önlemdir.
- **Rate Limiting:** `/login` üzerinde IP başına 60 saniyede 5 deneme sınırı (`RateLimiter` sınıfı) ile kaba kuvvet engellenmiştir.
- **Parola Özetleme:** Werkzeug PBKDF2 + sabit-zamanlı doğrulama ile kullanıcı enumeration ve timing saldırıları kapatılmıştır.
- **Oturum Sertleştirilmesi:** `HttpOnly`, `SameSite=Lax`, üretimde `Secure` çerez bayrakları ile XSS-tabanlı çerez hırsızlığı ve CSRF azaltılmıştır.
- **Güvenlik Başlıkları:** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Content-Security-Policy: default-src 'self'`, `Referrer-Policy: no-referrer` tüm cevaplara eklenmiştir.
- **BFLA Koruması:** `/admin/...` durum değiştiren uç noktalar `@admin_required` ile korunarak dikey yetki yükseltme engellenmiştir.
- **Birim Testleri:** `test_cases.py` içinde 31 test ile IDOR'un her iki modda (VULNERABLE / SECURE) beklenen davranışı CI sürecine bağlanmıştır; böylece olası bir gerileme (regression) anında yakalanır.

SAYFA 10 — SONUÇ VE KAYNAKÇA

10.1. Sonuç

Bu çalışmada, OWASP API Security Top 10 (2023) listesinin birinci sırasında yer alan **Broken Object Level Authorization (BOLA / IDOR)** zafiyeti; bir dijital fatura yönetim sistemi simülasyonu üzerinden uçtan uca incelenmiştir. Aynı uygulama, tek bir mod anahtarı ile zafiyetli ve güvenli sürümleri arasında geçişlendirilebilecek şekilde tasarlanmış; böylece zafiyetin nasıl ortaya çıktığı, nasıl sömürüldüğü ve nasıl giderildiği yan yana, ölçülebilir biçimde ortaya konmuştur. Burp Suite Repeater aracılığıyla yapılan sömürü, yalnızca URL yolundaki sayısal kaynak referansının değiştirilmesinin, sistem genelindeki tüm finansal verileri açığa çıkarmaya yettiğini somut olarak göstermiştir.

Çalışmanın en önemli teknik bulgusu, IDOR zafiyetlerinin **kod kalitesinden değil, mimari kararlardan** doğduğudur. Geliştirici, `@login_required` dekoratörünün yetkilendirme kontrolü yaptığı yanlıgısına düştüğünde, çözüm tek satır kod ekleyerek değil; **Authentication ile Authorization katmanlarını bilinçli olarak ayıran** bir tasarım dilini benimseyerek elde edilebilir. Bu çalışmada bu ayırım; `login_required`, `admin_required` ve `owner_required` dekoratörlerinin birbirinden bağımsız çalıştığı bir mimaride somutlaştırılmıştır.

İkinci olarak, **Güvenli Yazılım Geliştirme Yaşam Döngüsü (Secure Software Development Life Cycle — SSDLC)** prensiplerinin önemi bir kez daha doğrulanmıştır. SSDLC; gereksinim analizinden başlayarak tasarım, kodlama, test, dağıtım ve bakım aşamalarının her birine güvenlik faaliyetleri (tehdit modellemesi, güvenli kod denetimi, statik/dinamik analiz, sızma testi, bug bounty) yerleştirir. Bu çalışmada, tasarım aşamasında "her erişim isteğinde sahiplik doğrulanır" prensibinin politika olarak benimsenmesi; geliştirme aşamasında merkezî bir `check_object_ownership` fonksiyonunun yazılması; test aşamasında `test_cases.py` içinde IDOR senaryolarının otomatik testlere bağlanması; ve dağıtım sonrası süreçte güvenlik loglamasının SIEM entegrasyonu için hazırlanması, SSDLC pratiğinin örnek bir uygulamasını oluşturmaktadır. Sonuç olarak güvenlik, kodun tamamlanmasından sonra eklenen bir özellik değil; tasarımın ilk gününden itibaren içine örülen bir mühendislik niteliğidir. Bu rapordaki teknik bulgular, IDOR ve benzeri Broken Access Control zafiyetlerinin yalnızca disiplinli bir SSDLC ile sürdürülebilir biçimde önlenileceğini doğrulamaktadır.

10.2. Kaynakça

[1] OWASP Foundation, "**OWASP Top 10:2021 — A01:2021 Broken Access Control**", OWASP, 2021. Erişim: https://owasp.org/Top10/A01_2021-Broken_Access_Control/

- [2] OWASP Foundation, "**OWASP API Security Top 10 — 2023: API1:2023 Broken Object Level Authorization**", OWASP, 2023. Erişim: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>
- [3] OWASP Foundation, "**Insecure Direct Object Reference Prevention Cheat Sheet**", OWASP Cheat Sheet Series, 2024. Erişim: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
- [4] PortSwigger Ltd., "**What is IDOR (Insecure Direct Object Reference)? Tutorial & Examples**", Web Security Academy, 2024. Erişim: <https://portswigger.net/web-security/access-control/idor>
- [5] PortSwigger Ltd., "**Using Burp Suite Repeater for Manual API Testing**", PortSwigger Documentation, 2024. Erişim: <https://portswigger.net/burp/documentation/desktop/tools/repeater>
- [6] National Institute of Standards and Technology (NIST), "**SP 800-162 — Guide to Attribute Based Access Control (ABAC) Definition and Considerations**", NIST, 2014.
- [7] National Institute of Standards and Technology (NIST), "**SP 800-218 — Secure Software Development Framework (SSDF) Version 1.1**", NIST, 2022.
- [8] FIRST.Org, "**Common Vulnerability Scoring System v3.1: Specification Document**", Forum of Incident Response and Security Teams, 2019. Erişim: <https://www.first.org/cvss/v3.1/specification-document>
- [9] MITRE Corporation, "**CWE-639: Authorization Bypass Through User-Controlled Key**", Common Weakness Enumeration, 2024. Erişim: <https://cwe.mitre.org/data/definitions/639.html>
- [10] MITRE Corporation, "**CWE-284: Improper Access Control**", Common Weakness Enumeration, 2024. Erişim: <https://cwe.mitre.org/data/definitions/284.html>
- [11] Kişisel Verileri Koruma Kurumu (KVKK), "**6698 Sayılı Kişisel Verilerin Korunması Kanunu**", Resmî Gazete, 2016. Erişim: <https://www.kvkk.gov.tr/>
- [12] European Parliament and Council, "**General Data Protection Regulation (GDPR) — Regulation (EU) 2016/679**", Article 32 — Security of Processing, Official Journal of the European Union, 2016.
- [13] Pallets Projects, "**Flask Web Development Framework — Official Documentation, v3.0**", 2024. Erişim: <https://flask.palletsprojects.com/>
- [14] Pallets Projects, "**Werkzeug Security Helpers — generate_password_hash, check_password_hash**", Werkzeug Documentation, 2024. Erişim: <https://werkzeug.palletsprojects.com/en/latest/utils/#module-werkzeug.security>

[15] Aydemir, B. & Sarıduman, F., "**IDOR Güvenlik Simülasyonu — Kaynak Kod Deposu**", GitHub, 2026. Erişim: <https://github.com/aydemirbrkay/IDOR>